SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

CO1-ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Vutukuri Venkata Durga Nagasai Ratan Reddy	**Reg. No.:**	192324172
Section / Batch:	AI&DS	**Date:**	10-10-2026

Code of Conduct

*I **Vutukuri Venkata Durga Nagasai Ratan Reddy(192324172)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: V.V.D.Nagasai Ratan Reddy

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

```
import re

resumes = [
    """
    Name: Rahul Sharma
    Email: rahul@gmail.com
    Mobile: 9876543210
    Skills: Python, Java, SQL, Machine Learning
    Experience: 3 years
    """,
    """
    Name: Priya Kumar
    Email: priya@gmail.com
    Mobile: 9123456780
    Skills: Java, SQL, NLP
    Experience: 4 years
    """,
    """
    Name: Arjun Reddy
    Email: arjun@gmail.com
    Mobile: 9988776655
    Skills: Python, SQL, NLP
    Experience: 1 year
    """
]

skills_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]

eligible_candidates = []

for resume in resumes:
    name = re.search(r"Name:\s*(.+)", resume)
    email = re.search(r"[\w.-]+@[\w.-]+\.\w+", resume)
    mobile = re.search(r"\b[6-9]\d{9}\b", resume)
    exp = re.search(r"Experience:\s*(\d+)\s*years?", resume, re.I)

    skills = [
        skill for skill in skills_list
        if re.search(r"\b" + re.escape(skill) + r"\b",
                     resume, re.I)
    ]

    experience = int(exp.group(1)) if exp else 0

    profile = {
```

```
        "Name": name.group(1).strip() if name else "Not found",
        "Email": email.group() if email else "Not found",
        "Mobile": mobile.group() if mobile else "Not found",
        "Skills": skills,
        "Experience": experience
    }

    print("\nCandidate Profile")
    print("-----------------")
    for key, value in profile.items():
        print(key + ":", value)

    if experience >= 2 and "Python" in skills:
        eligible_candidates.append(profile)

print("\nEligible Candidates")
print("-------------------")

for candidate in eligible_candidates:
    print(candidate["Name"], "-", candidate["Experience"], "years")
```

Sample Output:

```
...
    Candidate Profile
    -----------------
    Name: Rahul Sharma
    Email: rahul@gmail.com
    Mobile: 9876543210
    Skills: ['Python', 'Java', 'SQL', 'Machine Learning']
    Experience: 3

    Candidate Profile
    -----------------
    Name: Priya Kumar
    Email: priya@gmail.com
    Mobile: 9123456780
    Skills: ['Java', 'SQL', 'NLP']
    Experience: 4

    Candidate Profile
    -----------------
    Name: Arjun Reddy
    Email: arjun@gmail.com
    Mobile: 9988776655
    Skills: ['Python', 'SQL', 'NLP']
    Experience: 1

    Eligible Candidates
    -------------------
    Rahul Sharma - 3 years
```

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

```
import re

products = [
    "Apple iPhone 15",
    "Samsung Galaxy S24",
    "Apple MacBook Air",
    "Dell Laptop",
    "HP Laptop",
    "Wireless Headphones",
    "Apple Watch",
    "Samsung TV",
    "Bluetooth Speaker"
]

keyword = input("Enter search keyword: ")
search_type = input(
    "Choose search type (exact/prefix/suffix/partial): "
).lower()

if search_type == "exact":
    pattern = r"\b" + re.escape(keyword) + r"\b"
elif search_type == "prefix":
    pattern = r"\b" + re.escape(keyword)
elif search_type == "suffix":
    pattern = re.escape(keyword) + r"\b"
elif search_type == "partial":
    pattern = re.escape(keyword)
else:
    print("Invalid search type")
    pattern = None

if pattern:
    matches = [
        product for product in products
        if re.search(pattern, product, re.IGNORECASE)
    ]

    print("\nMatching Products:")
    if matches:
        for product in matches:
            print(product)
    else:
        print("No products found.")

    print("\nSearch Report")
    print("Keyword:", keyword)
```

```
print("Search Type:", search_type)
print("Total Matching Products:", len(matches))
```

output:

```
Enter search keyword: apple
Choose search type (exact/prefix/suffix/partial): prefix

Matching Products:
Apple iPhone 15
Apple MacBook Air
Apple Watch

Search Report
Keyword: apple
Search Type: prefix
Total Matching Products: 3
```

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

```
import re

reg_no = input("Enter Student Register Number: ")
email = input("Enter Institutional Email: ")
course = input("Enter Course Code: ")
semester = input("Enter Semester (1-8): ")
mobile = input("Enter Mobile Number: ")

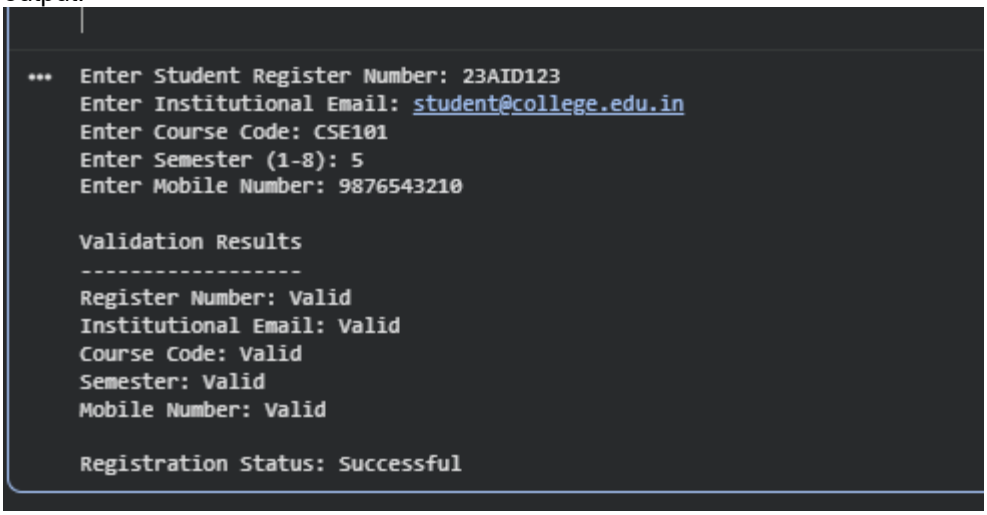
valid_reg = re.fullmatch(r"\d{2}[A-Z]{2,3}\d{3}", reg_no)
valid_email = re.fullmatch(r"[\w.-]+@[\w.-]+\.edu\.in", email)
valid_course = re.fullmatch(r"[A-Z]{2,4}\d{3}", course)
valid_semester = re.fullmatch(r"[1-8]", semester)
valid_mobile = re.fullmatch(r"[6-9]\d{9}", mobile)

print("\nValidation Results")
print("------------------")

print("Register Number:", "Valid" if valid_reg else "Invalid")
print("Institutional Email:", "Valid" if valid_email else "Invalid")
print("Course Code:", "Valid" if valid_course else "Invalid")
print("Semester:", "Valid" if valid_semester else "Invalid")
print("Mobile Number:", "Valid" if valid_mobile else "Invalid")

if all([valid_reg, valid_email, valid_course,
        valid_semester, valid_mobile]):
    print("\nRegistration Status: Successful")
else:
    print("\nRegistration Status: Failed")
    print("Please correct the invalid fields.")
```

output:



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							

2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____